



INFRA TESTER

Automação de Teste do fluxo: cadastro de demandas

Documento de Arquitetura de Sistemas

Versão 1.0

Elaborado por: Mickeias Charles de Oliveira Paiva (Estagiário SUPTI)

Julho de 2026

Histórico da Revisão

Data	Versão	Descrição	Autor
29/07/2026	1.0	Criação do documento com base na arquitetura implementada do projeto (robot/, backend/, frontend/, desktop/).	Mickeias Charles

Sumário

Histórico da Revisão	2
Sumário	2
1. Introdução.....	4
2. Desenho do Escopo.....	5
3. Visão Geral do Sistema.....	5
4. Estrutura de Pastas	6
5. Motor de Automação — robot/cadastrar_pca.robot	6
5.1 Forma de invocação.....	6
5.2 Fluxo de alto nível.....	7
5.3 Keywords auxiliares.....	7
5.4 Variáveis — o contrato com backend e desktop	7
5.5 Cuidados de segurança já resolvidos	8
6. Interface Web — backend/ e frontend/	8
6.1 Backend (FastAPI)	8
6.1.1 Ciclo de vida de uma execução.....	8
6.2 Frontend (React + Vite + TypeScript)	9
7. Interface Desktop — desktop/ (PySide6).....	9
7.1 Fluxo de telas	9
7.2 Execução do robô — diferença em relação ao backend.....	9
7.3 Persistência e caminhos	9
7.4 Empacotamento (build.sh)	10
8. Como Estender o Projeto.....	10
8.1 Adicionar um campo novo ao wizard.....	10
8.2 Adicionar um passo novo ao wizard	10
8.3 Se o layout do SISCORP mudar.....	10
9. Ambiente Necessário.....	10
9.1 Como rodar cada componente.....	11
Backend.....	11
Frontend.....	11
Desktop	11
Teste isolado do .robot.....	11

10. Diagnóstico de Problemas Conhecidos	11
11. Segurança — Regras que Não Devem Ser Quebradas.....	12
12. Ferramentas e Tecnologias.....	12
13. Referências	12

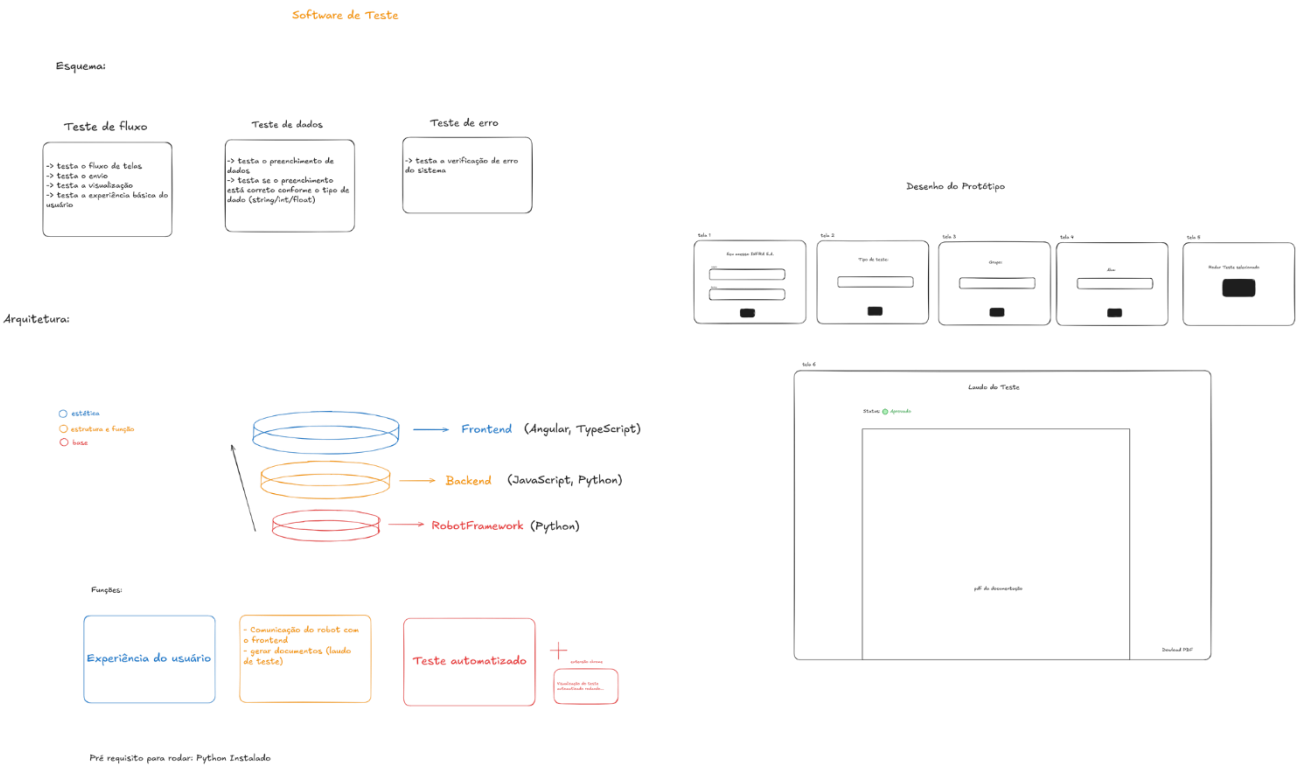
1. Introdução

Este documento descreve a arquitetura de software do sistema de automação de cadastro do SISCORP PCA (Plano de Contratações Anual) da INFRA S.A. O objetivo é registrar, de forma autodidata, o funcionamento completo do sistema, de modo que qualquer pessoa sem contexto prévio consiga entendê-lo, executá-lo localmente, depurar problemas comuns e estendê-lo com segurança.

O SISCORP não expõe nenhuma API própria: toda interação com o sistema é feita por meio de formulários web em navegador. Diante dessa restrição, a única via viável para automatizar o cadastro de uma nova demanda no PCA é a automação de navegador (RPA), simulando as ações de um usuário humano preenchendo o wizard de cadastro.

Este é um projeto vivo: novos campos, passos e integrações devem ser incorporados ao longo do tempo, e este documento deve ser atualizado sempre que houver uma mudança arquitetural relevante (novo componente, mudança de fluxo, nova dependência de ambiente), preferencialmente na mesma tarefa em que a mudança é feita.

2. Desenho do Escopo



3. Visão Geral do Sistema

O núcleo da automação é um script Robot Framework + Selenium (`robot/cadastrar_pca.robot`), responsável por fazer login no SISCORP e preencher o wizard de 5 passos de cadastro de demanda. Esse motor já existia antes deste projeto.

Sobre esse motor, foram construídas duas interfaces independentes para que uma pessoa alimente os dados sem precisar editar o arquivo `.robot` manualmente:

- Web (frontend/ + backend/) — formulário React servido por um backend FastAPI, pensado para múltiplos usuários acessando via navegador, com um servidor central executando o robô;
- Desktop (desktop/) — aplicativo PySide6 standalone, com o mesmo wizard e a mesma lógica, mas sem servidor: login, wizard, disparo do robô e histórico rodam no processo local do usuário. Pensado para ser distribuído como um único `.exe` em uma máquina Windows.

As duas interfaces são independentes entre si — nenhuma depende da outra — mas compartilham o mesmo arquivo de automação (`robot/cadastrar_pca.robot`) e o mesmo conjunto de variáveis. Caso o SISCORP mude de layout e o `.robot` precise de ajuste, a correção vale simultaneamente para as duas interfaces.

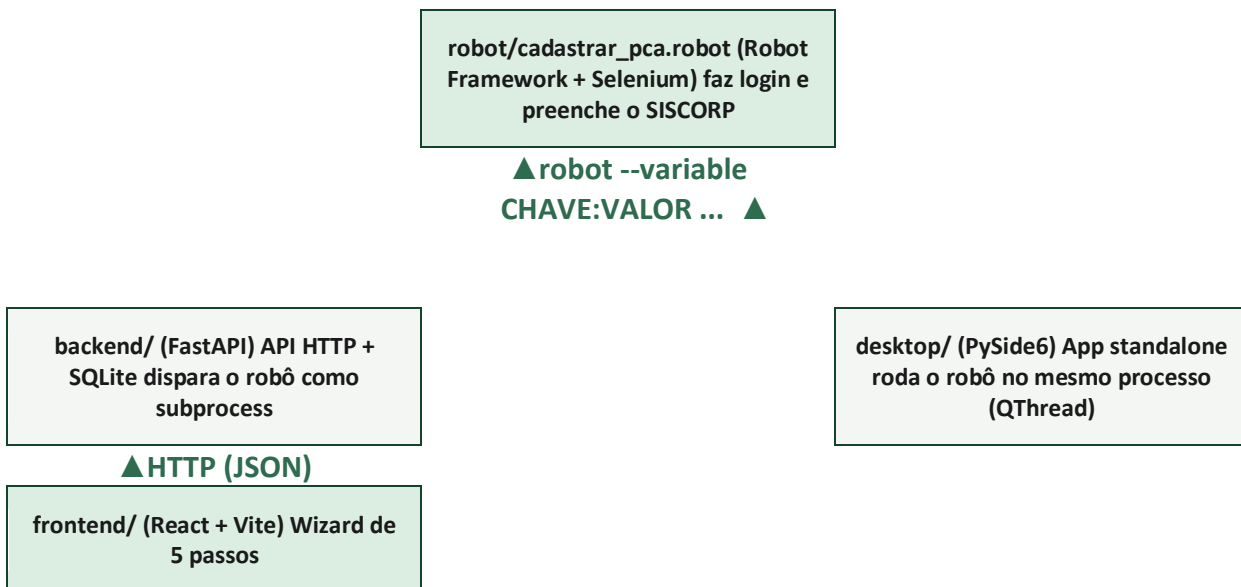


Figura 1 — Visão geral da arquitetura: robô de automação como núcleo compartilhado, com duas interfaces independentes (Web e Desktop).

4. Estrutura de Pastas

```

teste_siscorp/
├── robot/
│   └── cadastrar_pca.robot      # único arquivo de automação
├── backend/
│   ├── app/
│   │   ├── main.py            # FastAPI app, CORS, healthcheck
│   │   ├── config.py          # caminhos (ROBOT_FILE, RUNS_DIR, DB_PATH)
│   │   ├── database.py         # engine SQLAlchemy + sessão
│   │   ├── models.py          # tabela `executions` (ORM)
│   │   ├── schemas.py         # modelos Pydantic (validação)
│   │   ├── robot_runner.py     # dispara `robot` como subprocess
│   │   └── routers/executions.py # endpoints HTTP
│   ├── executions.db          # SQLite (fora do git)
│   └── runs/
├── frontend/
│   ├── src/
│   │   ├── App.tsx            # shell + rotas (react-router)
│   │   ├── api.ts             # client HTTP para o backend
│   │   ├── types.ts           # tipos TS espelhando schemas do backend
│   │   ├── pages/
│   │   └── components/steps/  # Step1..Step5 do wizard
│   └── package.json
├── desktop/
│   ├── main.py                # entry point (QApplication)
│   └── app/
│       └── config.py           # caminhos, com suporte a modo congelado
├── (PyInstaller)
│   ├── models.py              # dataclasses equivalentes aos schemas do backend
│   ├── db.py                  # SQLite local (sem SQLAlchemy)
│   ├── robot_runner.py        # roda robot.run(...) in-process, em QThread
│   ├── login_widget.py        # tela de login
│   ├── execucao_atual_widget.py # console ao vivo + laudo pós-execução
│   ├── main_window.py         # janela principal, navegação, sessão
│   └── wizard/
│       ├── nova_demanda_widget.py # orquestra os 6 passos do wizard
│       └── steps.py            # Step1..Step6 (Qt widgets)
├── build.sh                   # empacota com PyInstaller
├── execucoes.db               # SQLite local (fora do git)
├── runs/
└── dist/SiscorpPCA/           # gerado pelo build.sh

```

Figura 2 — Estrutura de diretórios do projeto.

5. Motor de Automação — robot/cadastrar_pca.robot

Este é o único componente que efetivamente enxerga e interage com o SISCORP. As interfaces Web e Desktop atuam como "controle remoto": montam uma lista de variáveis e solicitam ao Robot Framework que execute esse arquivo.

5.1 Forma de invocação

```

robot --variable CHAVE1:VALOR1 --variable CHAVE2:VALOR2 ... \
  --outputdir <pasta_de_saida> \
  robot/cadastrar_pca.robot

```

A senha nunca é passada por `--variable`, pois apareceria em logs de processo. Ela é fornecida por meio da variável de ambiente `SISCORP_SENHA`, lida pela keyword `Obter Senha SISCORP`.

5.2 Fluxo de alto nível

O caso de teste `Cadastrar Nova Demanda no PCA (SISCORP)` executa, em sequência: login no sistema, início do cadastro de demanda e o preenchimento dos cinco passos do wizard (`Dados Básicos`, `Itens de Contratação`, `Dados da Contratação`, `Previsão de Duração` e `Previsão de Desembolso`), finalizando com a validação de cadastro com sucesso e o fechamento do navegador.

A keyword `Executar Etapa` roda cada passo com `Run Keyword And Ignore Error`: se um passo falhar, o aviso é registrado em log e a execução segue para o próximo passo, em vez de abortar o teste inteiro. Isso é proposital — um campo que o SISCORP mudou de lugar não deve impedir o preenchimento dos demais passos. A keyword final `Validar Cadastro com Sucesso` é quem decide se o teste, como um todo, passou ou falhou.

5.3 Keywords auxiliares

Keyword	Finalidade
Preencher Campo React	Digita valor em input/textarea controlado por React, disparando os eventos input/change esperados pelo framework (um Input Text padrão do Selenium não atualiza o state do React)
Selecionar Opcao Dropdown / Abrir Dropdown Pelo Label / Marcar Opcao Visivel Por Texto	Lidam com combobox customizados (não são um <code><select></code> HTML nativo)
Preencher Data Via Datepicker Tolerante / Selecionar Dia No Calendario	Preenchimento de datas via datepicker próprio do SISCORP
Preencher Campo Monetario Por Texto Visual	Campos de valor em R\$ com máscara
Clicar Com JS / Clicar Nativo Depois JS	Fallback: clique nativo do Selenium às vezes não registra em elementos cobertos por overlay; força o clique via <code>Execute Javascript</code>
Fechar Overlays	Fecha modais/toasts que ficam sobre campos
Diagnosticar Tela Bloqueada	Executada quando <code>Validar Cadastro com Sucesso</code> não confirma sucesso; tira screenshot e registra pistas para depuração manual

5.4 Variáveis — o contrato com backend e desktop

Toda variável abaixo possui um valor padrão de teste dentro do arquivo `.robot`, mas backend e desktop sempre o sobrescrevem via `--variable`. Um campo novo no wizard sempre deve se tornar uma dessas variáveis (ver seção 8).

Variável	Preenchida em
NAVEGADOR	Chrome ou headlesschrome (definido pelo checkbox "headless")
USUARIO	Login
STATUS_CONTRATACAO, EVENTO, ANO_PCA, AREA_DEMANDANTE, DESCRICAO_OBJETO, JUSTIFICATIVA	Passo 1 — Dados Básicos
ACAO, PLANO_ORCAMENTARIO, TIPO_NATUREZA, NATUREZA_DESCRICAO, STATUS_SUBITEM, TIPO_SUBITEM, CODIGO_SUBITEM, DESCRICAO_SUBITEM, UNIDADE_MEDIDA, PRECO_UNITARIO, QUANTIDADE	Passo 2 — Itens de Contratação

Variável	Preenchida em
TIPO_LICITACAO, DATA_ASSINATURA, DATA_ENTREGA_DOC, OBJETIVO_ESTRATEGICO, PRIORIDADE, SIGILOSO	Passo 3 — Dados da Contratação
VIGENCIA_MESES	Passo 4 — Previsão de Duração
TIPO_DESEMBOLSO, PARCELA_ANUAL, VALOR_PARCELA_UNICA (opcional), ANO_DESEMBOLSO, MES_DESEMBOLSO, VALOR_MENSAL_DESEMBOLSO	Passo 5 — Previsão de Desembolso

A senha (SISCORP_SENHA) é a única exceção: é fornecida por variável de ambiente, nunca por `--variable`.

5.5 Cuidados de segurança já resolvidos

- Senha nunca aparece em log: Realizar Login no Sistema e Obter Senha SISCORP envolvem toda manipulação da senha com Set Log Level NONE e restauração do nível anterior. Sem esse wrapping, a senha vazaria em texto puro no log.html/output.xml.
- `--no-sandbox / --disable-dev-shm-usage` em Open Browser: o Chrome se recusa a abrir com sandbox ativo quando o processo roda como usuário root (comum em WSL/containers).
- Set Window Size 1920 1080 em vez de Maximize Browser Window: este último depende de um fallback via CDP (Runtime.evaluate) que quebra em versões recentes do Chrome headless.

6. Interface Web — backend/ e frontend/

6.1 Backend (FastAPI)

Todos os endpoints ficam sob o prefixo `/api/executions`.

Método	Rota	Função
POST	<code>/api/executions</code>	Recebe um DemandaPCA (JSON), cria uma Execution (status pending) e dispara o robô em background
GET	<code>/api/executions</code>	Lista todas as execuções, mais recente primeiro
GET	<code>/api/executions/{id}</code>	Detalhe de uma execução
GET	<code>/api/executions/{id}/console</code>	Console ao vivo (se em execução) ou tail salvo (se finalizada)
GET	<code>/api/executions/{id}/report</code>	report.html gerado pelo Robot Framework
GET	<code>/api/executions/{id}/log</code>	log.html com screenshots de falha
GET	<code>/api/health</code>	Healthcheck

6.1.1 Ciclo de vida de uma execução

- POST `/api/executions` grava no SQLite com status="pending" e o payload da demanda (sem a senha);
- `robot_runner.start_execution` inicia uma `threading.Thread`, sem bloquear a requisição HTTP;
- A thread monta as variáveis e chama `subprocess.Popen(["robot", ...], env={..., "SISCORP_SENHA": senha})`;
- Cada linha de stdout/stderr do robô é gravada em tempo real em `runs/<id>/console.log`;
- Ao término, status vira success ou failed, `log_tail` guarda as últimas ~60 linhas e `finished_at` é registrado;
- `report.html`, `log.html` e `output.xml` ficam em `runs/<id>/`, servidos via `FileResponse`.

O backend requer Chrome/Chromium instalado na máquina que o executa — é essa máquina que efetivamente abre o navegador e acessa o SISCORP, não o navegador de quem preenche o formulário React.

6.2 Frontend (React + Vite + TypeScript)

Rota	Página	Função
/nova (e /)	NovaDemanda.tsx	Wizard de 5 passos + credenciais; chama api.criarExecucao ao final
/execucoes	Execucoes.tsx	Tabela com histórico (via api.listarExecucoes)
/execucoes/:id	ExecucaoDetalhe.tsx	Console ao vivo (poll), botões para relatório/log

A camada de API (src/api.ts) é um client HTTP fino, sem biblioteca externa. O arquivo types.ts espelha manualmente os schemas Pydantic do backend — não há geração automática de tipos: um campo alterado em schemas.py precisa ser replicado manualmente aqui.

7. Interface Desktop — desktop/ (PySide6)

Mesmo domínio do par frontend/backend (mesmo wizard, mesmas variáveis do .robot), porém com arquitetura bem diferente: não há cliente/servidor, tudo roda no mesmo processo Qt. É pensado para distribuição como um único executável Windows, sem que o usuário final precise instalar Python, subir um servidor ou abrir portas.

7.1 Fluxo de telas

- LoginWidget → usuário, senha e opção headless;
- NovaDemandaWidget → 6 passos: Dados Básicos, Itens de Contratação, Dados da Contratação, Previsão de Duração, Previsão de Desembolso e Revisão e Execução;
- ExecucaoAtualWidget → console ao vivo durante a execução e laudo (status + indicadores) ao final, com acesso a relatório e log.

O login é feito uma vez por sessão do aplicativo (diferente da Web, onde a senha é digitada a cada execução). A senha fica apenas em memória (classe Credenciais em app/models.py) e nunca é gravada em disco. Ao clicar em "Executar no SISCORP", o widget grava a execução no SQLite local, inicia uma RobotRunnerThread e a janela principal navega para a tela de execução.

7.2 Execução do robô — diferença em relação ao backend

O backend chama robot como processo separado (subprocess.Popen). O desktop chama robot.run(...) in-process, dentro de uma QThread (classe RobotRunnerThread). Essa abordagem evita depender de robot estar no PATH do sistema — essencial para o .exe empacotado, que carrega o Robot Framework embutido — e permite capturar cada linha de saída via um objeto file-like que grava em console.log e emite um sinal Qt por linha, atualizando a interface em tempo real.

7.3 Persistência e caminhos

A persistência local usa sqlite3 puro (sem SQLAlchemy/Pydantic), com um schema equivalente ao do backend. O arquivo app/config.py é o ponto mais delicado do desktop, pois se comporta de forma diferente entre o modo de desenvolvimento e o modo empacotado:

Item	Modo dev (python main.py)	Modo empacotado (sys.frozen)
ROBOT_FILE	../robot/cadastrar_pca.robot	robot/cadastrar_pca.robot dentro do bundle do PyInstaller (sys._MEIPASS)
RUNS_DIR / DB_PATH	Dentro de desktop/	Ao lado do executável (sys.executable.parent)

Qualquer alteração em config.py deve ser testada nos dois modos: rodando python main.py e reempacotando com build.sh.

7.4 Empacotamento (build.sh)

```
pyinstaller --noconfirm --clean --name SiscorpPCA \
  --add-data "../robot/cadastrar_pca.robot:robot" \
  --collect-all robot \
  --collect-all SeleniumLibrary \
  --collect-all selenium \
  main.py
```

As três flags `--collect-all` são obrigatórias: o Robot Framework carrega bibliotecas como `BuiltIn/SeleniumLibrary` dinamicamente por nome, não por import estático, e o analisador de dependências do `PyInstaller` não enxerga esse padrão sozinho. Sem essas flags, o `.exe` compila normalmente mas falha em runtime ao importar `robot.libraries.BuiltIn`. Rodar `build.sh` em Linux gera um binário Linux; para gerar o `.exe` Windows é necessário executá-lo em uma máquina/VM Windows, já que o `PyInstaller` não faz cross-compile.

8. Como Estender o Projeto

8.1 Adicionar um campo novo ao wizard

Um campo novo sempre toca quatro lugares, pois não há geração de código entre as camadas:

- `robot/cadastrar_pca.robot`: nova ``${VARIABEL}` em `*** Variables ***` (com valor padrão de teste) e uso na keyword do passo correspondente;
- Backend: `schemas.py` (novo campo no submodelo `Pydantic`) + `robot_runner.py` (`_variables_for`);
- Frontend: `types.ts` + o `StepN...tsx` correspondente;
- Desktop: `app/models.py` (`dataclass`) + `variaveis_robot()` + `wizard/steps.py`.

Recomenda-se localizar uma variável parecida nos quatro componentes antes de começar — copiar o padrão de um campo existente do mesmo tipo (texto, combobox, data, monetário) é mais seguro que escrever do zero.

8.2 Adicionar um passo novo ao wizard

- No `.robot`: nova keyword `Preencher Passo N - <Nome>` + chamada em `Executar Etapa`;
- Frontend: novo `StepN.tsx`, registrado em `NovaDemanda.tsx` e no `StepIndicator.tsx`;
- Desktop: nova classe `StepN...` em `wizard/steps.py`, adicionada à lista de construção em `nova_demanda_widget.py`.

8.3 Se o layout do SISCORP mudar

O `.robot` é o único componente que precisa mudar — nem backend, nem frontend, nem desktop sabem como o SISCORP é construído internamente; eles apenas passam variáveis. O procedimento é: ajustar a keyword afetada, rodar `robot --dryrun` para validar a sintaxe, testar manualmente e reempacotar o desktop (`./build.sh`), já que ele carrega uma cópia embutida do arquivo.

9. Ambiente Necessário

- Python 3.10+ (backend e desktop possuem seus próprios `venv/requirements.txt`);
- Node 18+ (frontend);
- Chrome ou Chromium na máquina que efetivamente executa o robô — o Selenium Manager baixa um `chromedriver` compatível automaticamente, desde que haja acesso à internet na primeira execução;
- Acesso de rede a `siscorp-pca-des.infrasa.gov.br` (VPN da INFRA S.A., se aplicável) — sem isso, o robô abre o navegador mas nunca alcança a página de login.

9.1 Como rodar cada componente

Backend

```
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev      # dev server em http://localhost:5173
npm run build    # build de produção em dist/
```

Desktop

```
cd desktop
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python main.py
```

Em WSL2 com WSLg (Windows 11), a janela do desktop aparece diretamente na área de trabalho Windows sem configuração extra, já que o WSLg cuida das variáveis DISPLAY/WAYLAND_DISPLAY automaticamente.

Teste isolado do .robot

```
# valida sintaxe sem executar de verdade
python3 -m robot --dryrun --outputdir /tmp/out robot/cadastrar_pca.robot

# execução real, sobrescrevendo variáveis específicas
SISCORP_SENHA="sua_senha" python3 -m robot \
  --variable USUARIO:seu.usuario \
  --variable NAVEGADOR:headlesschrome \
  --outputdir /tmp/out \
  robot/cadastrar_pca.robot
```

10. Diagnóstico de Problemas Conhecidos

Sintoma	Causa raiz	Correção
Importing library 'robot.libraries.BuiltIn' failed (só no .exe empacotado)	PyInstaller não inclui bibliotecas carregadas dinamicamente pelo Robot Framework	--collect-all robot/SeleniumLibrary/selenium no build.sh
SessionNotCreatedException: Chrome instance exited	Chrome recusa abrir com sandbox ativo rodando como root	--no-sandbox e --disable-dev-shm-usage em Open Browser
Mesmo erro acima, mesmo com --no-sandbox	Binário do Chrome baixado pelo Selenium Manager corrompido/incompleto	Apagar ~/.cache/selenium e instalar o Chrome oficial via pacote do sistema
WebDriverException: unknown command: 'Runtime.evaluate'	Maximize Browser Window usa fallback via CDP incompatível com Chrome headless recente	Trocado por Set Window Size 1920 1080

Sintoma	Causa raiz	Correção
Senha aparecendo em texto puro em log.html/output.xml/console	Robot Framework loga automaticamente o retorno de \${var}= Keyword e o valor lido de campos de senha	Set Log Level NONE / restauração em volta de toda manipulação da senha

11. Segurança — Regras que Não Devem Ser Quebradas

- A senha do SISCORP nunca é persistida em disco em nenhum dos três componentes. Ela só existe: (a) na requisição HTTP/tela de login; (b) na variável de ambiente SISCORP_SENHA do processo do robô durante aquela execução; (c) em memória, na sessão do app desktop;
- O .robot nunca deve logar a senha. Qualquer keyword nova que manipule \${SENHA}/\${senha_final} precisa do mesmo wrapping Set Log Level NONE;
- backend/executions.db, backend/runs/, desktop/execucoes.db e desktop/runs/ ficam fora do controle de versão (.gitignore), pois acumulam histórico de execuções reais que pode conter dados sensíveis do PCA.

12. Ferramentas e Tecnologias

Camada	Tecnologia	Descrição
Automação	Robot Framework + Selenium	Motor de RPA que interage com o SISCORP via navegador
Backend (Web)	FastAPI + SQLAlchemy + SQLite	API HTTP, persistência das execuções e disparo do robô como subprocesso
Frontend (Web)	React + Vite + TypeScript	Wizard de 5 passos e histórico de execuções
Desktop	PySide6 (Qt) + sqlite3	Aplicativo standalone com o robô rodando in-process
Empacotamento	PyInstaller	Geração do executável Windows standalone
Ambiente	WSL2 / Ubuntu / WSLg	Ambiente de desenvolvimento em máquina Windows sem privilégios administrativos

13. Referências

Robot Framework, Documentação oficial, link: <https://robotframework.org/>, acessado em 2026.

SeleniumLibrary for Robot Framework, Documentação oficial, link: <https://robotframework.org/SeleniumLibrary/>, acessado em 2026.

FastAPI, Documentação oficial, link: <https://fastapi.tiangolo.com/>, acessado em 2026.

PySide6 (Qt for Python), Documentação oficial, link: <https://doc.qt.io/qtforpython/>, acessado em 2026.

PyInstaller, Documentação oficial, link: <https://pyinstaller.org/>, acessado em 2026.